Austin Fairbanks
11/01/2025

Optimizations in Edge AI: Techniques and Challenges in Deploying Machine Learning at the Edge

Research Methods Summary

Literature on edge machine learning is broad, often raising questions about where the line between cloud and edge computing should be drawn; in this synthesis, we define it as enabling on-device inference without requiring transmission to another computational source. In real-world applications, however, achieving low latency and maintaining a small physical footprint while still delivering effective outputs are integral to building usable products. This reveals a hidden complexity in how edge machine learning must be applied outside theoretical contexts. These challenges lead to a central question: how are data, model, and system-level optimization strategies being used to overcome the computational and resource constraints of deploying machine learning models at the edge?

Most literature on edge machine learning (ML) surveys software optimization methods that can be applied across a wide range of hardware platforms and assumes that readers have general machine learning knowledge. While this work is useful, these sources lack a concrete definition of what edge computing is and provide little information about the purposefully engineered devices that enable it. This led to the exploration of papers that not only clearly define the boundary between cloud and edge but also provide specific hardware examples that employ system-level optimizations.

Overall, this synthesis draws on a set of sources that each bring a different angle to edge ML. I started by grounding myself in the basics of edge computing. For that, I used the IBM Edge AI article, which gives a clear and direct explanation of what counts as edge computing and how it differs from cloud setups. From there, I used the UMass Libraries database to look for broad surveys. I searched “edge machine learning survey” and selected the survey by Wang and Jia because it covers the main data, model, and system components of the field and is widely referenced. It gave me a solid overview of the major challenges and techniques.

To go deeper into specific areas, I kept searching through the UMass Libraries database. Using terms like “training on the edge,” I found Khouas et al., which focuses on training-side optimizations, especially around data ingestion and practical constraints. Since the general surveys did not go far enough on hardware and system design, I then used Google to look for articles discussing software and hardware challenges together. Searches such as “edge AI hardware platforms” and “embedded AI system optimization” led me to Garcia-Perez et al., which breaks down recent hardware architectures used for edge ML. Finally, to include the software and compiler side, I searched for “edge ML compiler optimization” and selected Jouini et al. because it concentrates on compiler-level techniques and system-software interactions. I then cross referenced these documents with ISBN’s in the UMass database to establish increased reliability and to obtain further information. Taken together, these sources give both broad coverage and detailed insight. They each fill in a different part of the picture, from definitions and high-level surveys to hardware, training, and software optimizations.

Finding 1: Introduction, Benefits, and Challenges

Edge machine learning is difficult to strictly define. It is not constrained to a specific hardware platform or application, but instead depends on a few key characteristics. First, edge ML must include some level of non-network processing, where information is stored and processed on a physical device rather than exclusively in the cloud. Second, it must interact with at least one physical sensor or information source, such as a camera or distance sensor, to connect computation to the external environment [1]. Finally, it must be able to act intelligently on observed data, satisfying the basic requirement of using machine learning to drive decisions or behavior. These core components still support a wide range of applications and introduce tradeoffs when selecting techniques to improve performance.

Edge ML also faces numerous challenges and conflicting objectives that often require experimentation to resolve, depending on the specific application. Primarily, practitioners face the constraint of limited resources, such as compute and memory. Additionally, as chips become more powerful, they also tend to consume more energy, which edge devices seek to avoid in mobile applications [5]. Moreover, there are additional concerns with data on the edge, such as data privacy and hidden data interdependence. Sensor data is usually noisy and temporal, meaning it has inherent relationships over time, which becomes difficult for models that assume independent and identically distributed samples. Then, when attempting to mitigate these problems by connecting to cloud servers or other devices, systems must rely on potentially insecure network protocols. This increases the risk that data will be leaked to external parties, often containing sensitive information at the edge [4]. All of these challenges introduce inherent tradeoffs, which are discussed in the following sections.

Finding 2: Data Enhancement and Learning Techniques

Garbage in, Garbage out (GIGO) refers to the importance of data in machine learning algorithms; if a model is trained with bad data, it is nearly impossible to obtain a satisfactory result [5]. This makes data preprocessing techniques of the utmost importance in machine learning, ensuring that the training data is meaningful and represents the same underlying distribution in deployment. In edge ML, these data practices are even more vital, as we usually deal with unlabeled, noisy, and high-volume sensor data [4]. Due to these challenges, both data-centric and learning-centric techniques are used to improve model performance at the edge.

One significant data-focused technique is feature compression. This involves removing redundant and irrelevant features from the data, often improving accuracy, reducing complexity, and enhancing interpretability. Features may also be extracted or combined into new, more relevant metrics [5].

After the data is preprocessed, there are many learning techniques that can also help address edge ML challenges. Transfer learning, used widely across machine learning, is a popular method that involves taking a compressed base model trained on a general dataset and fine-tuning the last few output layers for a specific application. These techniques find their main use case in embedded convolutional networks used for image processing, such as a ResNet-50 backbone [3]. While transfer learning can improve model efficiency on a single device, there also exist methods designed for distributed learning, used when working with a large set of edge sensors across a collection of different devices. Federated learning is a technique that transforms model learning into a distributed process by storing a shared global model across devices and incrementally training it on-device without requiring storage of sensitive data, as single-model approaches do [4]. While these techniques may suffer at certain scales, they both provide alternatives for training that allow for decreased model size while limiting performance degradation.

Finding 3: Model-Level Optimizations

Generalized machine learning applications, often hosted on cloud servers, are typically not optimized for resource use. When differences in computation time are on the order of milliseconds and inferences are not executed frequently, there is little pressure to optimize [5]. However, in limited compute environments, several model-structure techniques have been developed to decrease the workload.

To start with, there are a few general techniques that can be used to decrease model complexity regardless of structure. The most common is model quantization, a pre- or post-training process that reduces the precision of activations and outputs to decrease storage requirements [3]. In addition, instead of lowering precision, model pruning can be used. Pruning removes redundant layers or parameters that contribute little to the model's overall predictive performance [5]. The primary drawback of these methods is that they reduce model capacity, or the range of data distributions the model can represent. Although this sometimes improves generalization, it can also lead to performance degradation when the model becomes oversimplified [5].

While these general techniques are widely used to adjust model size and performance, they do not leverage domain knowledge that could further improve efficiency. To supplement them, many domain-specific architectures have been developed to create smaller yet highly effective models. For example, in image processing, methods such as ShuffleNet V2 reorganize color channels into subgroups to enable smaller-scale batch processing, which reduces model size [5]. In other domains, neural architecture search (NAS) has proven useful by applying optimization algorithms to evaluate different network structures. In edge ML, NAS typically explores specialized architectures like ShuffleNet rather than minor structural variants [5].

Finding 4: Specialized Edge Hardware and Systems

While hardware used at the edge varies greatly, a few main device families are commonly used for machine learning. The first family is the single-board computer (SBC), typically based on a system-on-chip (SoC) design. SBCs resemble personal computers but have all hardware components (RAM, CPU, GPU, etc.) located on a single printed circuit board rather than as separate modules. These machines can run full operating systems, essentially acting as minicomputers. Examples include the Raspberry Pi, NVIDIA Jetson Nano, and Google Coral Dev Board [2]. SBCs also have a smaller, less complicated cousin known as the microcontroller (MCU). An MCU plays a similar role, usually being paired with an input/output board like an SBC, but it provides much less CPU power and memory storage. A widely used device in this category is the ESP32, part of the Arduino family, which appears in many household appliances, toys, and vehicles. Finally, there exists another family called field-programmable gate arrays (FPGAs), which are widely used in highly parallel applications due to their ability to distribute computation effectively [2].

While SBCs may contain the most processing power of these devices, they are also the most power hungry when deployed. On mobile, battery-powered platforms, many practitioners therefore favor MCUs as the processor of choice. Inherently, this choice limits the types of models that can be run on these devices, because they are much less capable than their SBC counterparts. Even in cases where FPGAs provide computational gains through parallelism, distributed computation requires a larger physical footprint than any of the devices above. Table 1 from [5] (pg. 5) summarizes some of the capabilities of each chip.

Table 1. Comparison between device families.

Characteristics	SBC (SoC)	MCU	FPGA
Purpose	General purpose or more complex applications.	Low consumption applications.	Signal processing and general purpose logic.
CPU	Powerful CPU and usually x86 or ARM architecture.	Simple CPU with low power and performance.	No integrated CPU, uses gate arrays.
Components	RAM, SO, peripherals, etc.	RAM and peripherals.	Configurable gate matrix for hardware level design.
Computing capacity	High computing capacity, graphic processing and memory.	Limited computing capacity and memory.	Computational capacity and logic are highly configurable.
Energy consumption	Higher power consumption.	Lower power consumption.	Variable power consumption.
Flexibility	Less flex.	Less flexibility, focused on specific applications.	Highly flexible and configurable.
Programmability	Fully programmable, various OS and languages.	Programmable but limited languages.	Highly programmable.
Cost	Moderate-high cost.	Low cost.	High cost.

In addition to specialized hardware-side design for edge ML, there are specific libraries and frameworks that provide software-side optimizations. In a more general sense, libraries such as TensorFlow and PyTorch both provide “lite” or “mobile” models that are pretrained and compressed to fit on edge devices. These work well in many applications and provide cross-compatible models for any of the hardware types above [3]. Other approaches favor hardware-centric optimizations, such as Espressif’s deep learning library, which is designed for use only on ESP32-S3 and ESP32-P4 microcontrollers [6]. These are just a few examples of software-centric optimizations, as the landscape is changing rapidly.

Finding 5: Future Directions and Applications

Most commonly, edge ML is seen in autonomous vehicles, smart factories, security systems, and healthcare. Autonomous vehicles use larger but compressed object-detection computer vision models to map their surroundings in real time. In smart factories and security systems, data can be collected to identify a normal operating state and to train a model to detect anomalies, such as machine failures or unusual sensor activity. Finally, healthcare systems can run on person to monitor metrics such as blood pressure and administer vital medication when needed [5].

In future applications, edge machine learning can be used to encrypt sensitive data, standardize IoT devices, and provide quality control across all connected devices in an edge ecosystem [3] As hardware and software iteratively improve, the potential for new applications continues to expand.

Conclusion

Overall, research shows that there is no “one size fits all” approach to edge deployment optimization. It requires complex domain knowledge, application-specific performance enhancements, and deliberate tradeoffs between conflicting objectives such as power consumption, accuracy, and physical footprint. While there are some common techniques, such as data-centric feature compression and model-centric transfer learning, that inherently provide advantages, these are not applicable to all domains. Techniques that are more widely applicable, such as model pruning or quantization, require reduced precision and capacity to be effective. These trends extend to the system level, as SBCs may offer more computational power but at the expense of greater power consumption and larger physical footprints compared to MCUs. Even on the software side, many optimizations introduce a precision cost that is inherently built into the library. This requires practitioners to remain up to date with the most recent techniques in edge ML to ensure they achieve the best possible configuration for their specific applications

Works Cited

1. Michael Flinders and Ian Smalley. 2025. Edge AI versus cloud AI: what's the difference? Retrieved from https://www.ibm.com/think/topics/edge-vs-cloud-ai
2. Alberto Garcia-Perez, Raquel Miñón, Ana I. Torre-Bastida, and Estibaliz Zulueta-Guerrero. 2023. Analysing edge computing devices for the deployment of embedded AI. *Sensors*. Retrieved from https://www.mdpi.com/1424-8220/23/23/9495
3. Oussama Jouini, Khaoula Sethom, Abdessamad Namoun, Nouf Aljohani, Mohammed H. Alanazi, and Mohammad N. Alanazi. 2024. A survey of machine learning in edge computing: techniques, frameworks, applications, issues, and research directions. *Technologies* 12, 81. Retrieved from https://www.mdpi.com/2227-7080/12/6/81
4. Abderrahmane R. Khouas, Mourad R. Bouadjenek, Hacene Hacid, and Shailesh Aryal. 2024. Training machine learning models at the edge: a survey. *arXiv preprint*. arXiv:2403.02619. Retrieved from https://arxiv.org/abs/2403.02619
5. Xiaoliang Wang and Wei Jia. 2025. Optimizing edge AI: a comprehensive survey on data, model, and system strategies. *Journal of Systems Architecture*. Retrieved from https://arxiv.org/html/2501.03265v1
6. Espressif Systems. 2025. ESP-DL: deep learning library for ESP32 series. Retrieved from https://docs.espressif.com/projects/esp-dl/en/latest/getting_started/readme.html